Reviewer Pack — Minimal Order of Geometrically Independent Lattices Bounds C…

Entry d8d109f8949f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 15:44:25 UTC

Minimal Order of Geometrically Independent Lattices Bounds Communication Complexity for XOR

Entry ID: `d8d109f8949f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 15:44:25 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Geometry (Lattice Theory) **Field B** (complexity object): Boolean Function Complexity (Circuit Complexity of XOR)

Statement:

> For any N×N boolean matrix M with N ≤ 40, the communication complexity for computing the XOR of its rows is upper bounded by O(log^2(N) * log(min(|I|, N-|I|))), where |I| is the minimal number of independent lattices that generate M.

Rationale (proposer's reasoning):

> Lattice theory has been used to study communication complexity in the past, but not in conjunction with the geometric structure of independent lattices. This conjecture aims to bridge this gap by linking the geometric independence of lattices with the complexity of XOR computation, potentially revealing new insights into the limits of communication complexity.

Taxonomy category: `cg_kw_andreev` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8653b8dbf63cea43`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a boolean matrix M of size N×N (N in {10, 15, …, 40}), if the computed communication complexity for XOR is less than or equal to O(log^2(N) * log(min(|I|, N-|I|))) with Spearman rank correlation coefficient > 0.7 and all seeds produce a similar pattern.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.90	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.80	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: `ADJACENT_OK` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `lattice theory AND communication complexity XOR` - `Boolean function complexity XOR AND geometrically independent lattices` - `min order independent lattices AND bounds communication complexity XOR`

Top relevant hits considered: - [http://arxiv.org/abs/0710.4339v1] Heavy-Quark Masses from the Fermilab Method in Three-Flavor Lattice QCD - [http://arxiv.org/abs/1111.0981v1] Form factors for B to Kll semileptonic decay from three-flavor lattice QCD - [http://arxiv.org/abs/1308.1705v1] Rotating lattice - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/1407.6169v3] Multiplicative Complexity of Vector Valued Boolean Functions - [http://arxiv.org/abs/1112.2313v2] QBF-Based Boolean Function Bi-Decomposition - [http://arxiv.org/abs/2107.07540v3] Fast First-Order Algorithm for Large-Scale Max-Min Fair Multi-Group Multicast Beamforming - [http://arxiv.org/abs/1410.4915v3] Rankin-Selberg L-functions in cyclotomic towers, III

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=143.2s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 10 + (seed % 3) * 5  # Sweep through N in {10, 15, 20, 25, 30}

    def generate_matrix(n):
        return [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

    def is_independent(lattice, matrix):
        for i in range(len(matrix)):
            if any(all(matrix[i][j] == lattice[j][k] for k in range(len(matrix))) for j in range(i
                return False
        return True

    def find_minimal_lattices(matrix):
        lattices = []
        for i in range(1, len(matrix) + 1):
            for subset in combinations(range(len(matrix)), i):
                lattice = [[matrix[j][k] if k in subset else 0 for k in range(len(matrix))] for j
                if is_independent(lattice, matrix):
                    lattices.append(lattice)
```

```
        return min(lattices, key=len)

    def communication_complexity(n, min_lattice_size):
        return math.log2(n) ** 2 * math.log(min(min_lattice_size, n - min_lattice_size))

    matrix = generate_matrix(n)
    min_lattice = find_minimal_lattices(matrix)
    min_lattice_size = len(min_lattice)
    computed_complexity = communication_complexity(n, min_lattice_size)
    upper_bound = math.log2(n) ** 2 * math.log(min(min_lattice_size, n - min_lattice_size))

    return {
        "metric_name": "communication_complexity",
        "metric_value": computed_complexity,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": computed_complexity <= upper_bound,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample='not supported' first_failing_seed={first_failing
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_857f7cc6.py", line 62, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_857f7cc6.py", line 45, in run_trial
    computed_complexity = communication_complexity(n, min_lattice_size)
                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_857f7cc6.py", line 40, in communication
    return math.log2(n) ** 2 * math.log(min(min_lattice_size, n - min_lattice_size))
                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: math domain error
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means that the pre-registered support condition was not unambiguously met. | next: Re-run the test with proper error handling to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19509
2	preregistration	ollama_remote	glm4:latest	0	0	16347
3	novelty	ollama_remote	glm4:latest	0	0	9104
4	novelty	ollama_remote	glm4:latest	0	0	12469
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15587
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10710
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16200
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9725
9	judge	ollama_remote	glm4:latest	0	0	29142

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 138793 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d8d109f8949f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d8d109f8949f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d8d109f8949f.tar.gz` (if generated)